Team KAYA's game's development manual V1.1.0

This file includes a map for working with the game's files and scripts to prevent merge conflicts and duplicate code

How things are unified

General

1) Folder structure
2) Scripts structure
3) Code structure
4) GitHub Commits and Merge Conflicts

Specific

1) Textures

How to add elements to the preexisting systems

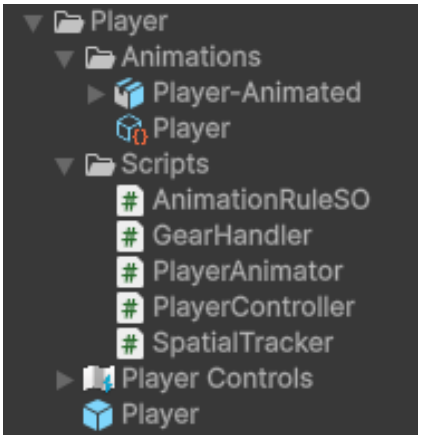
1) Add a weapon
2) Add an enemy

Where to add/modify existing systems (What manages What)

1) Entities' animations and visuals (EntityAnimator, PlayerAnimator)
2) Player's combat stats and values (PlayerStats)
3) Global world values and systems managing game during runtime (GameManager)
4) Enemies' shared behavior (EnemyController)
5) Weapons and their damage (WeaponSO, HeavyWeaponSO, LightWeaponSO)

Folder structure

Every major folder in the **Assets** folder represents an element of the game, for example…



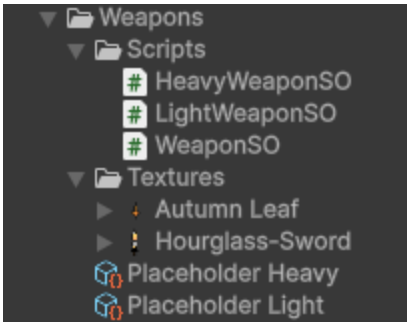
The **Player** folder contains the player prefab itself and the controls asset since the controls asset is only a single instance and not worth being put in a separate folder, and the player prefab is the reason the folder was created in the first place, serving as the **main element** of this folder, therefore it's put straight in the folder without adding a subfolder to it

On the other hand, the **Animations** subfolder contains the player's animations and sprites and the animation rule of the player, they are put in a separate folder because they **Serve** the main element, the **Player**, and are not a main element of the **Player**'s folder

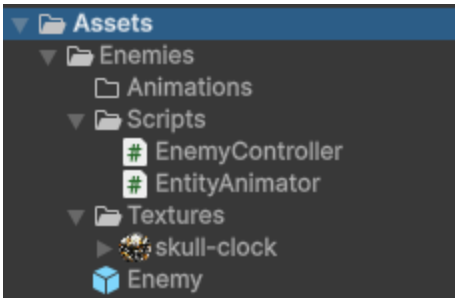
Same for the **Scripts** folder, the player requires multiple scripts to do multiple functions, therefore they are all put in a single scripts subfolder

All this is meant so when you get to modify an element of the game, you don't end up opening a file that explodes into lots of assets that you can't make a difference between, so when a developer gets to edit or add to the player's scripts for example, they know exactly where to go

Same goes for the rest of the elements of the game…



*Even though both weapons are ScriptableObjects and not prefabs, they are the **main** element of this folder, that's why they're not put in a subfolder*



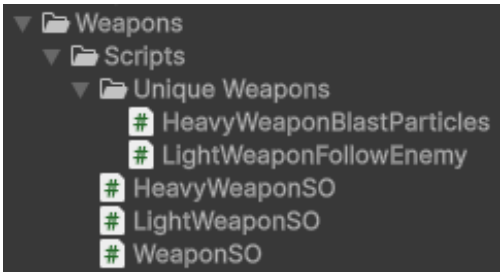
The Enemy prefab is inside the large folder itself and not in a subfolder, all other enemies' prefabs are added next to it

Scripts structure

Where to add a script

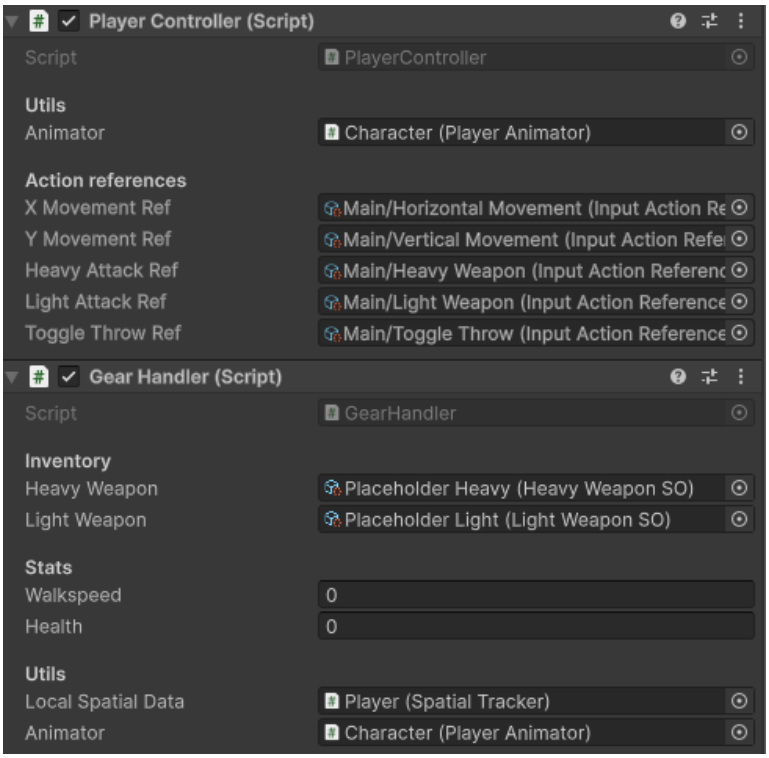
As long as the number of scripts is little or manageable, you may stack them all up in a single **Scripts** subfolder, if they start to become more difficult to go through, you may create a subfolder to separate them based on what they do or their type

For example, if you have multiple **ScriptableObject** scripts concerning a single element, put them all together in a subfolder inside the scripts folder and name it **ScriptableObjects**



If multiple scripts are variants of another script and/or inherit from it, group them up in a single folder like the example image here, this is mostly used for variants of **enemies, weapons and upgrades**

When to add a script



If you are writing a script that manages a lot of things at once (health, damage, movement, controls, visuals), consider splitting them apart into separate scripts *(Notice in figure 1 how PlayerStats manages the player's inventory and health while the PlayerController is managing only the player's controls and keybinds)*

If you are writing a script that has code that is required by two other scripts, consider putting it in a separate script as a **utility** *(Notice in figure 1 how both PlayerStats and PlayerController are using PlayerAnimator)*

When to NOT add a script

If the functionality you're willing to add already has a script that manages it (A new jump action for example), consider writing it in that script (PlayerController for example) alongside the existing code there

Refer to the **What manages What** section in the manual to know where you should add your next code or features

Code structure

When writing your code make sure to follow these rules

1) Variables are written like “playerHealth” where the first character is small and the next sentences’ characters are capitalized, and all sentences are close to each other
2) Classes, Enums and ScriptableObjects are written like “PlayerController” where the first character of every sentence is capitalized
3) Try to keep the names of variables and classes short
4) If any “if” statement is only applying a single line of code and this line is not too long, consider wrapping it up in a single line (*sorry, Yusuf)*, otherwise use curly brackets {}

```
// Animations
if (xMovementDir != 0 || yMovementDir != 0) Animator.state = EntityState.Walking;
else Animator.state = EntityState.Idle;
```

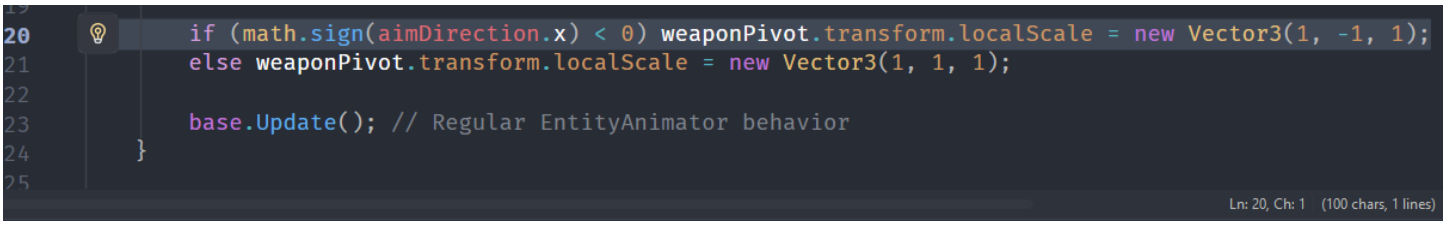
5) Separate elements that show up in the inspector with **Headers**
6) Set the elements that are only used by this script but are modifiable from the inspector as **[SerializeField] private** like the image below
7) Set the elements that are used by multiple scripts but not editable in the inspector as **[HideInInspector] public** like the image below
8) Add comments to your work either above or below it, so you and your team know why these lines are added here, you can also use comments for suggestions

```
[Header("Inventory")]
3 references
public HeavyWeaponSO heavyWeapon;
4 references
public LightWeaponSO lightWeapon;

[Header("Stats")]
2 references
public float walkspeed;
1 reference
public int health;

[Header("Utils")]
2 references
[SerializeField] private SpatialTracker localSpatialData;
1 reference
[SerializeField] private PlayerAnimator animator;
// For now we keep getting the camera from the animator and the mouse here
// We could write a third party script to track and return these values for both scripts independently, later though

2 references
[HideInInspector] public LocalWeaponsData localWeaponsData = new LocalWeaponsData();
```

9) If a function only has a single line of code, consider writing it like this

```
private void Update() => currentCooldown = Mathf.Max(0f, currentCooldown - Time.deltaTime);
// Or fixedDeltaTime? should it change according to time speed?
```

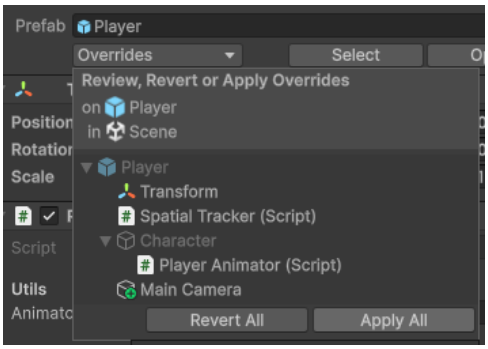
10) Define your functions as either **public** or **private** based on usage
11) If a line is longer than 110 characters, try to divide it into multiple lines



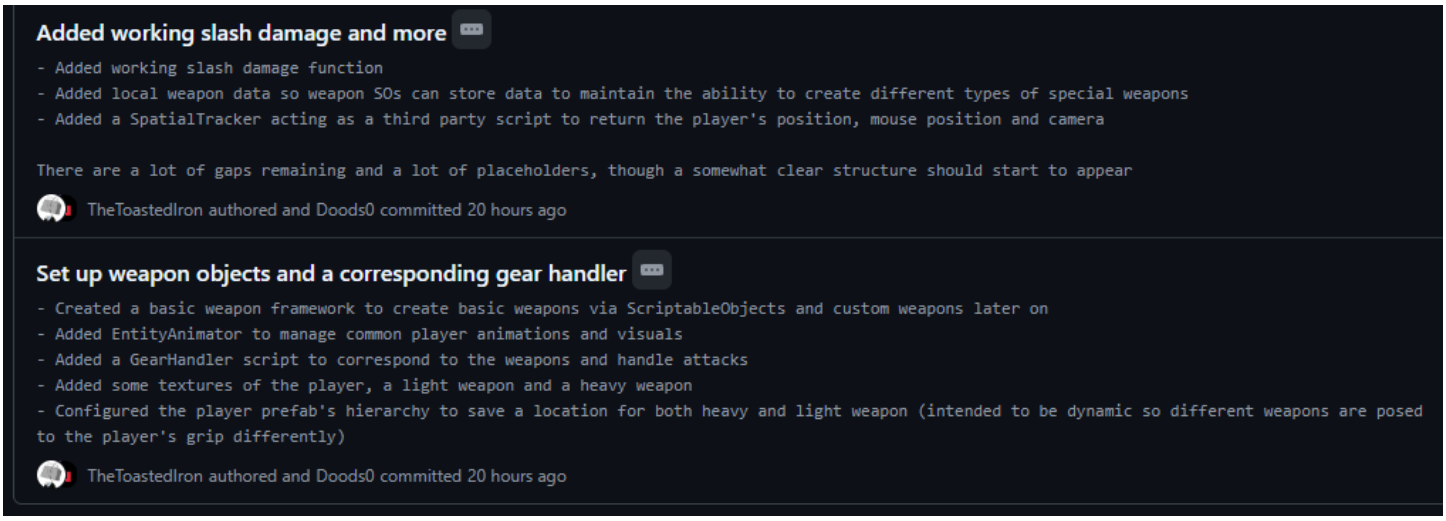
GitHub Commits and Merge Conflicts

When pulling requests for commits on GitHub make sure to follow these rules

1) When doing changes to the scene's elements for testing (Adding GameObjects, Adding prefabs with placeholder values), make sure you create a separate scene file for it as unity scenes are formatted as a single file, and modifying it is modifying the whole file which may lead to merge conflicts, name the scene after the feature you're testing



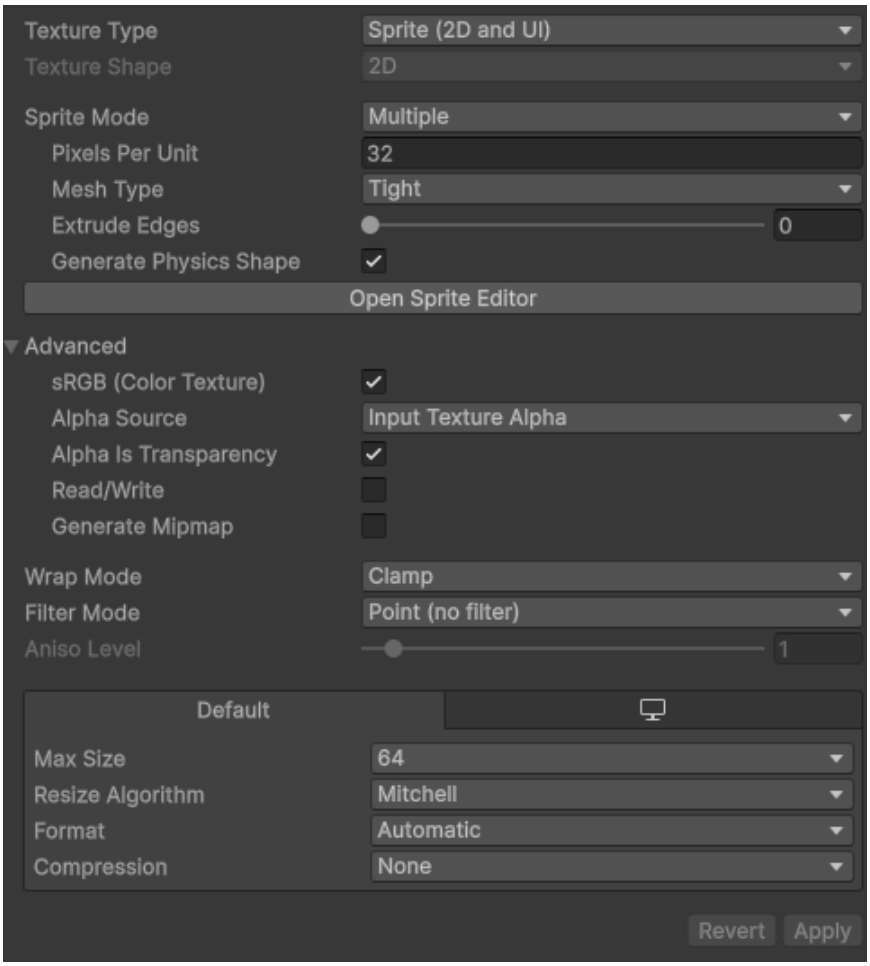
2) Make sure to apply your changes on any prefab that you edited if these changes are intended to be global
3) Please check your request good enough, it's not advisable to fill the commits tab with "Added this", "Removed this"
4) It's advisable that your commit contain multiple changes at once, don't just pull a request because you edited a value in a scene, **unless it's a bugfix, commit those as soon as possible no matter how small**
5) Make sure to describe most of the changes and additions you added and the purpose of them, no need to say all the details, details are described in the code as comments



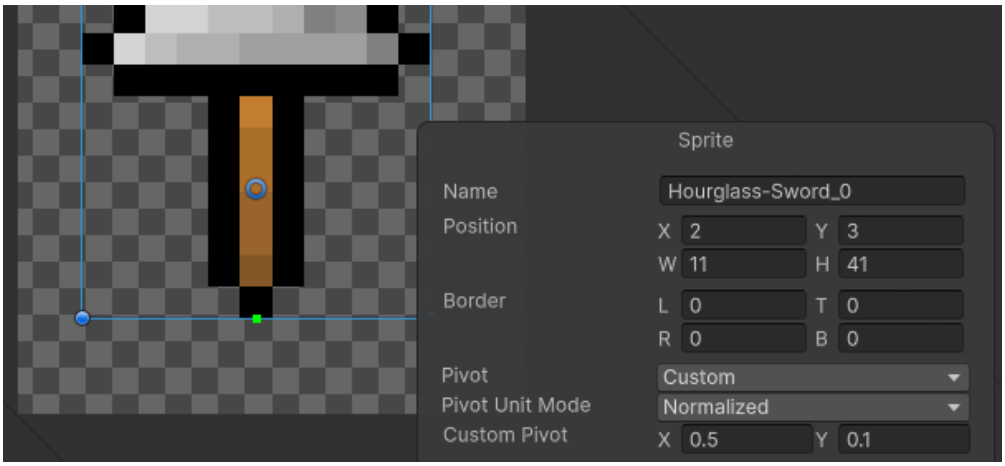
Textures

Pixel art textures MUST have the same **Pixels Per Unit** and **Extrude Edges** values in this image, they must also have the same **Filter Mode** value, and the **Max Size** value depends on the size of the texture, assign it to the least value possible so it's almost the same or greater than the value of the bigger dimension of the texture

(for example, a 24x32 texture needs to have this assigned to 32)

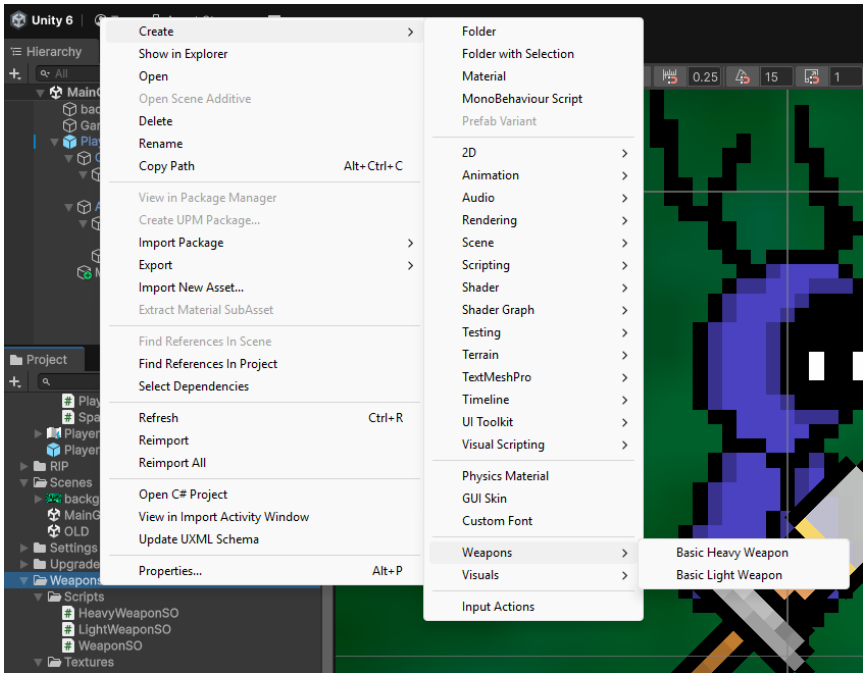


Weapons must have their **Custom Pivot** value set to the weapon's grip



How to Add a Weapon

Right click on the weapons folder, navigate through create → weapon → **Basic Heavy Weapon** or **Basic Light Weapon**



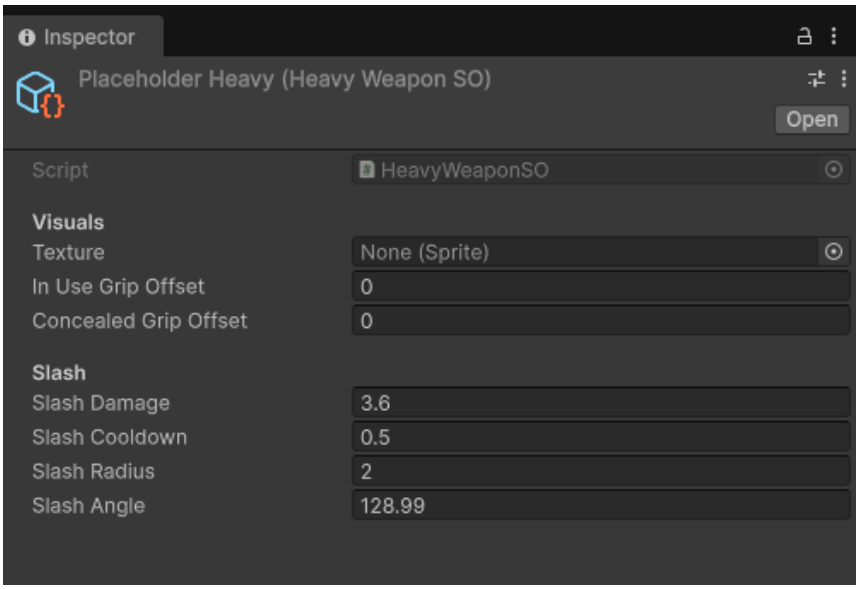
Texture defines the texture of the weapon

In Use Grip Offset defines the position of the weapon in the player's hand

Concealed Grip Offset defines the position of the weapon in the player's back

Slash Radius defines how far this weapon can melee hit

Slash Angle defines how wide the effective range of the attack of the weapon is

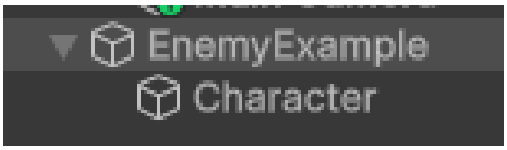


Cooldown defines the time between every hit

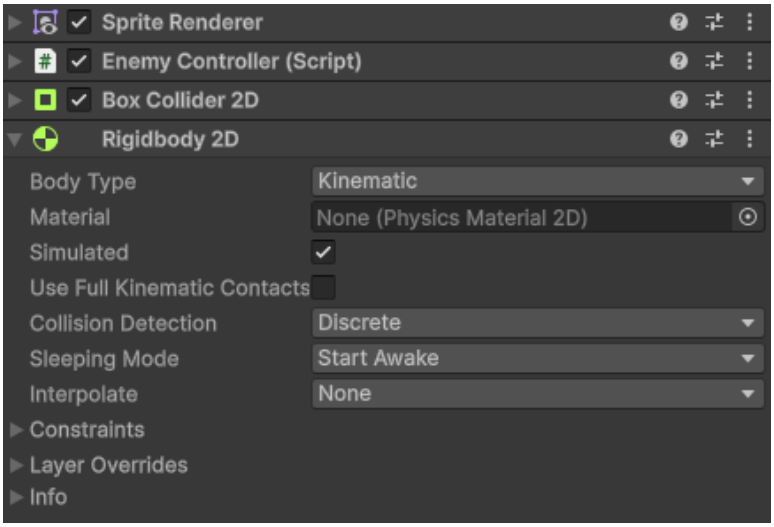
To know how to add **Custom/Special Weapons**, please refer to the [Weapons and their damage] section

How to Add an Enemy

To add a basic enemy, create a GameObject and another GameObject inside of it called **Character**

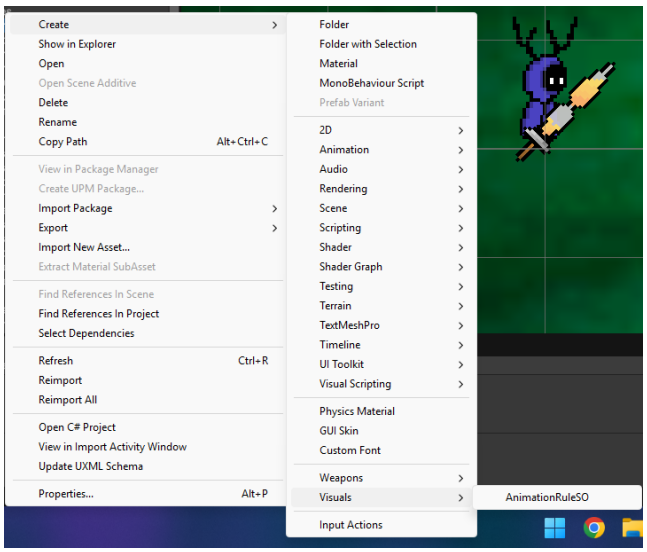


Make sure the parent GameObject has these components in it, **make sure the Rigidbody2D is set to Kinematic**, set up the **Box Collider** so it's almost in the size of the enemy, and set the references and values needed by the rest of the components and scripts



Make sure that the **Character** GameObject has these components

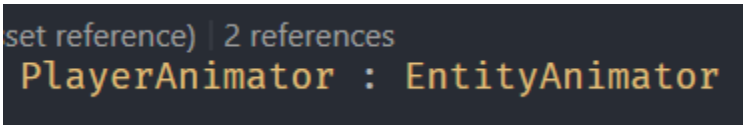
Drag and drop your .aseprite file inside the **Animations** folder, open the imported package and locate your imported animations, create a new **Animation Rule** and add the animations to it, then add it to the **EntityAnimator**



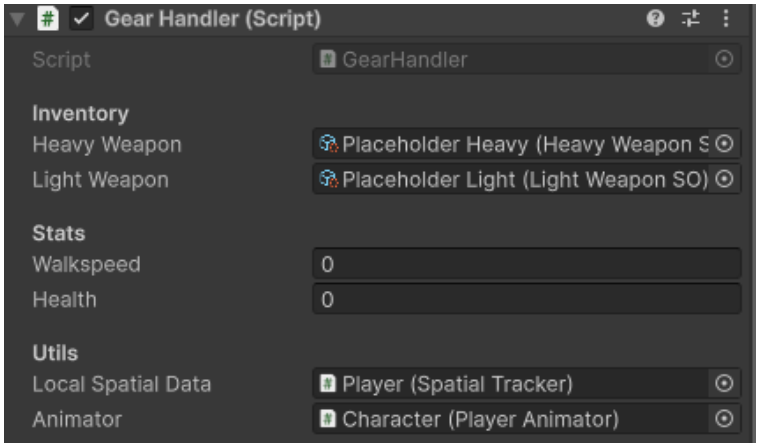
*To add a **Custom Enemy**, please refer to the [Enemies' shared behavior] section*

Entities' Animations and Visuals

Animations and visual effects for **Entities** are managed through a set of functions written in the **EntityAnimator** script located under the **Enemies** folder which manages stuff like switching animations and flipping characters

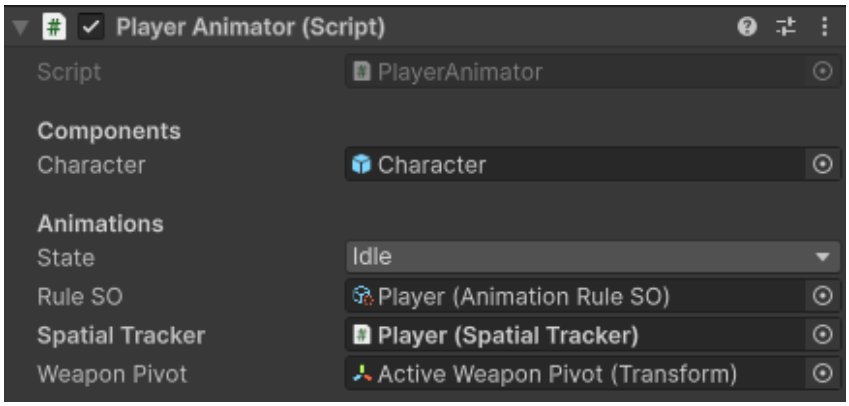


Visual effects for the player are managed by the **PlayerAnimator** script which inherits from the **EntityAnimator** script, meaning that this script includes all the functions in the EntityAnimator and more specific functions for the player like rotating the weapon towards the mouse location

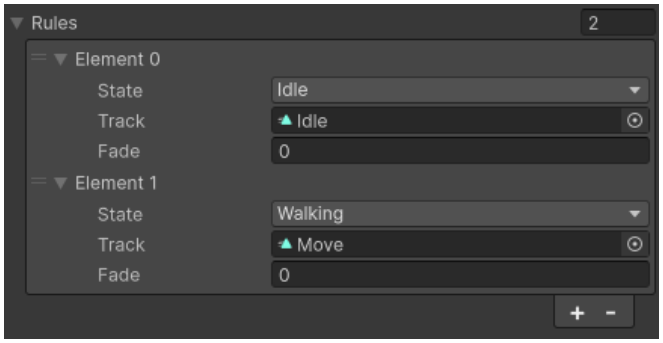


If the function to be added is passive and works all the time (Weapon following mouse for example) let the script itself execute it, if the function activates on action (Flashing on taking damage for example) execute the function from outside of the script with a reference *(notice how the PlayerStats has a reference to the animator script to run effects based on actions)*

Animations



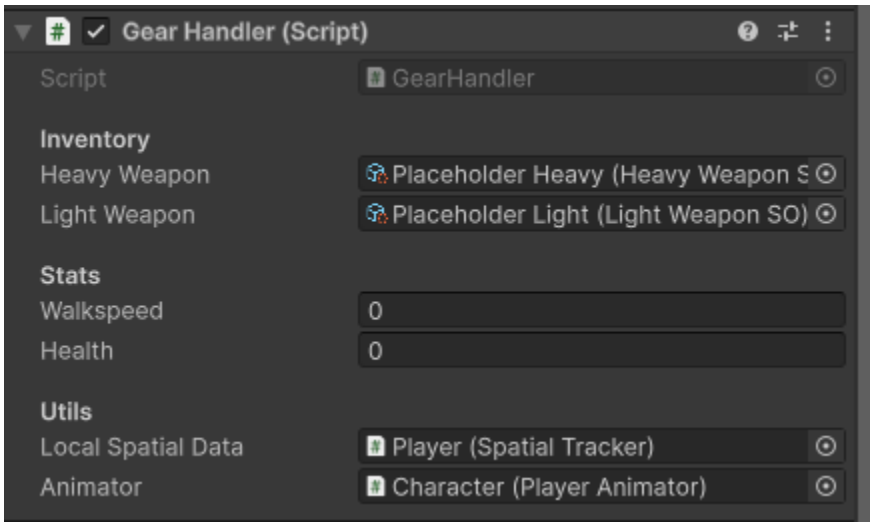
The Animator script requires a **Rule SO** parameter which is the **ScriptableObject** that holds the set of **Animation Rules** the entity will use



The Animator script runs animations based on the **State** Enum parameter, it looks around the **Rule SO** for the animation that holds the same value there

(In this example, the animator will run the "Idle" animation)

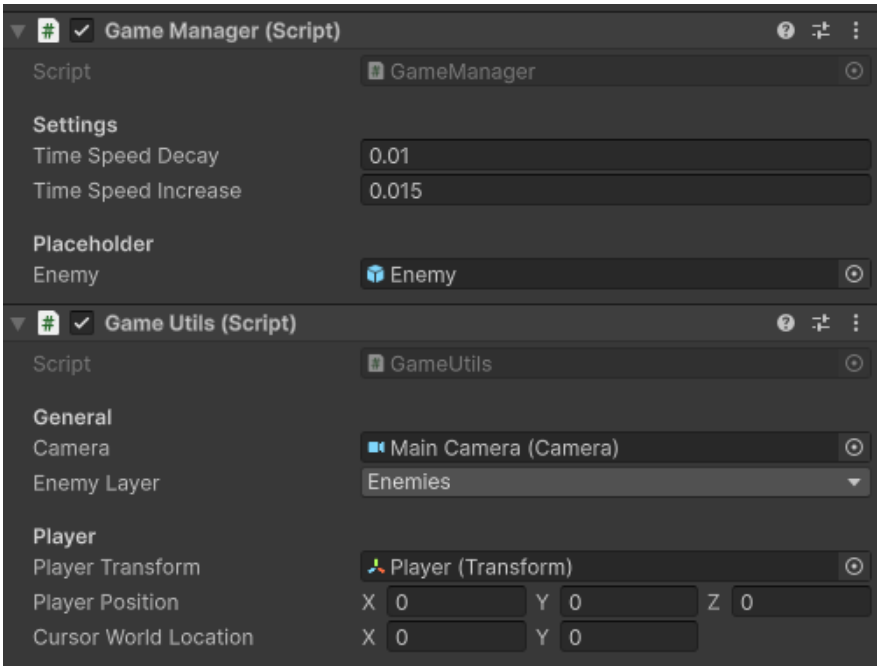
Player’s combat stats and values



Combat, Upgrades, Weapons and **Health** are all managed within the **PlayerStats** since all these stats affect each other, upgrades give buffs either to player’s stats like health or to the weapon’s damage, which is what this script is meant for, this script is still under construction but it’s meant to loop through all given upgrades to the player to apply their effects whether it’s a one-time effect like extra health or syncinc effects like damage increase relative to player movement or such

PlayerStats also manages weapons animations and visual effects using **PlayerAnimator**

Game Manager



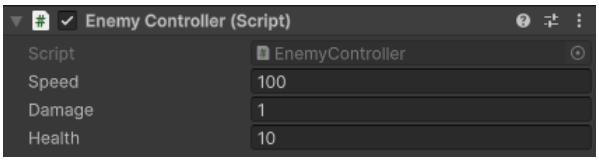
Although incomplete, this script is meant to manage all runtime concerns like enemy spawning, enemy pooling and increasing difficulty

It’s accompanied by a **GameUtils** script that holds the important global data of the game like the **player’s position** and **cursor world location** so it’s accessible around other scripts without having to drag and drop a reference to each script

This script is currently under development, if any of the mentioned features crowd the script then it’s advisable to slice them into multiple scripts

Enemies' shared behavior

All enemies have a unified behavior, **follow player and damage on touch**, this logic is meant to be held inside the **EnemyController** script *(Currently a work-in-progress)*



Adding Custom/Special Enemies

Adding custom enemies is meant to be done by creating custom Monobehavior scripts that inherit from the **EnemyController** script so all custom enemies follow the same base behavior

In order to add custom enemies for example, override the **FixedUpdate()** function inside the **EnemyController** with your own new code/checks, and add any extra functions necessary

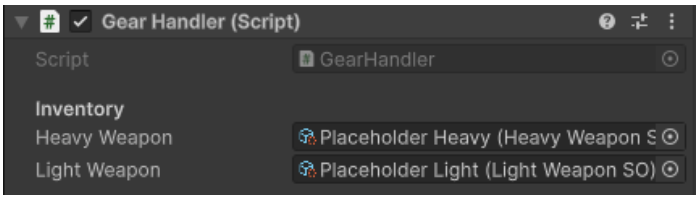
```
Unity Script | 0 references
public class CustomEnemyExample : EnemyController
{
    Unity Message | 2 references
    public override void FixedUpdate()
    {
        base.FixedUpdate(); // RUN THE DEFAULT BEHAVIOR IN ENEMYCONTROLLER

        // Add HERE
        // if player close enough stop moving and shoot bullets
    }
}
```

Weapons and Damage

Weapons are coded as **ScriptableObjects** with variable stats to ensure variety with minimal coding

Basic weapon variations are **HeavyWeaponSO** and **LightWeaponSO** that inherit from **WeaponSO** that holds all the common variables and functions among both types

Weapons are meant to be **ScriptableObjects** that get plugged in the **PlayerStats** that reads and manages their data



Adding Custom Weapons / Modifying base weapons

To add or modify to the weapons system, beware that the weapon is a **ScriptableObject**, which makes it unable to have an independent memory, you **cannot** save data inside the weapon's code, that's why you'll have to save it in a mono script, which will be the **PlayerStats** script itself, all the data concerning weapons will be stored in a class called **LocalWeaponData** that will be saved on the **PlayerStats** script and passed back and forth

```
2 references
public virtual float Slash(LocalWeaponsData weaponMemory)
{
    List<Collider2D> hitBuffer = weaponMemory.hitsBuffer;
    Vector3 playerPos = GameUtils.instance.playerPosition;
```

In case there are conflicts between the memory of the first and second weapon, we may create two instances of this class, one for each, however unification is advisable

```
#region Weapon's Memory
[Serializable]
2 references
public class LocalWeaponsData
{
    1 reference
    public readonly List<Collider2D> hitsBuffer = new List<Collider2D>();
    2 references
    public ContactFilter2D enemyFilter = new ContactFilter2D();
    // Moved the enemy filter from here to the mono script so it's reusable for knockback
}
#endregion
```